



7장 Shadows

김해울

용어 정리

- **Occluders** cast shadows onto **receivers**.
- A **hard shadow** is produced when a punctual light source with no area is used. A **soft shadow** is produced when a light source with area is used.
- **Umbra** – a fully shadowed region
- **Penumbra** – a partially shadowed region

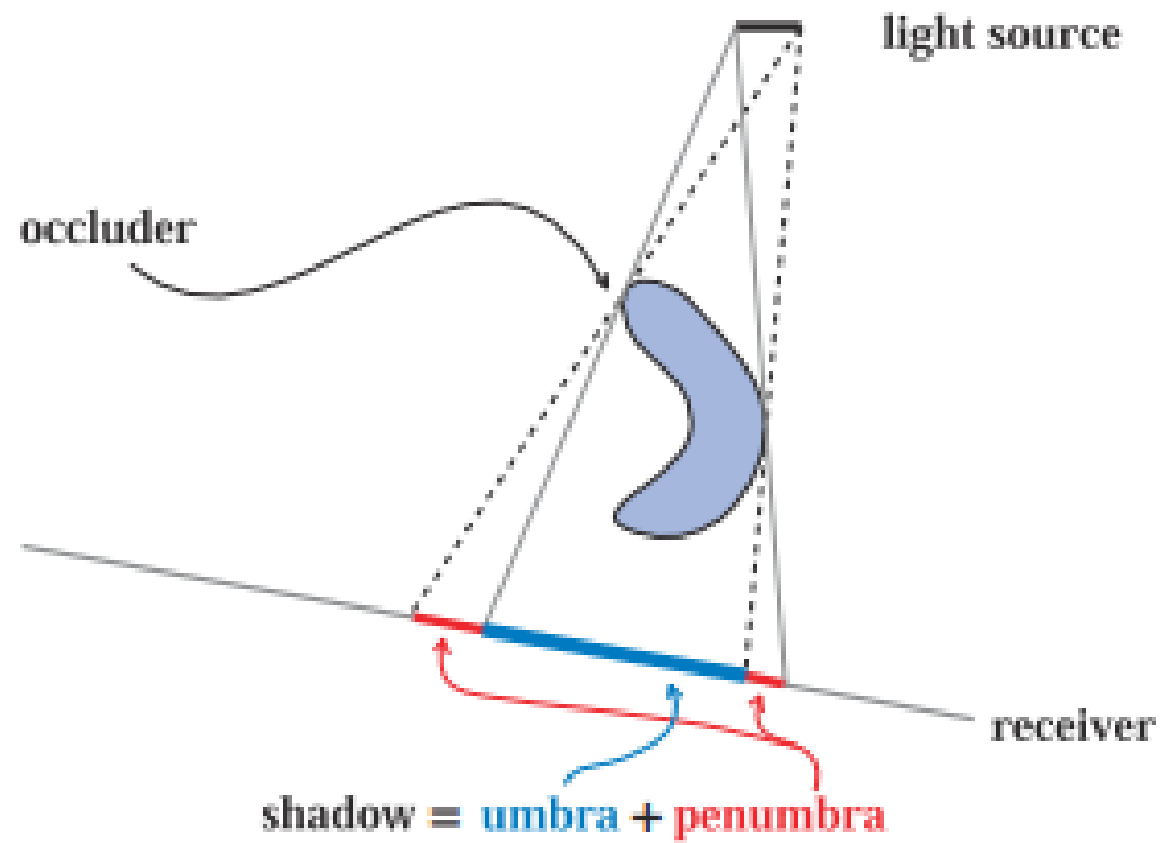
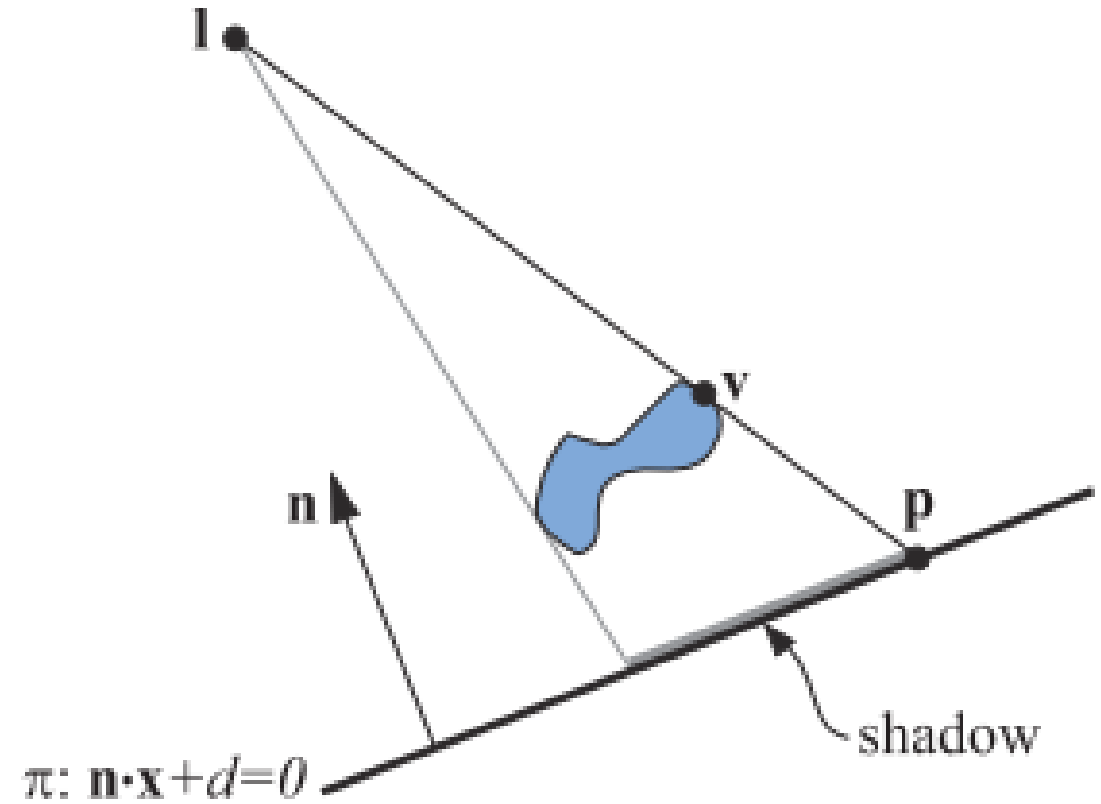
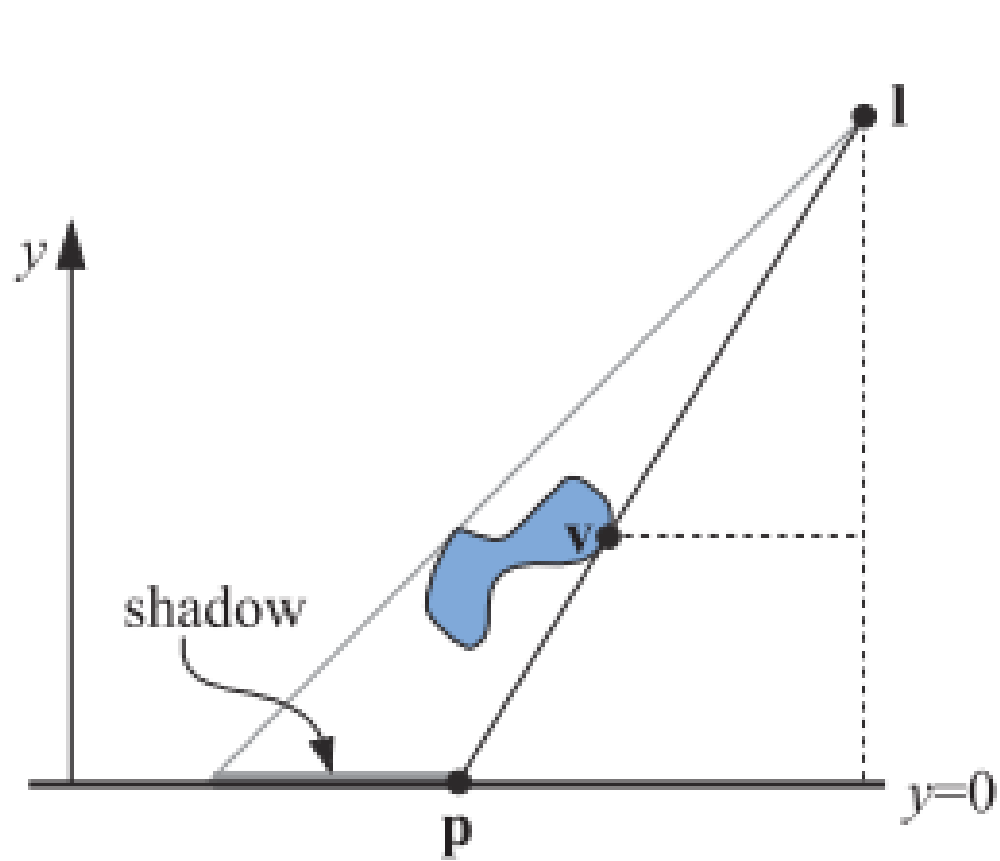


Figure 7.1



Planar Shadows – Projection Shadows

Figure 7.3

Three-dimensional object is rendered a second time to create a shadow.

Planar Shadows – Projection Shadows

- First, the projection matrix will be derived for the special case where the shadowed plane is $y = 0$, and then this result will be generalized to work with any plane.
- Left part of fig 7.3 – we get the following equation from similar triangles.
- $$\frac{p_x - l_x}{v_x - l_x} = \frac{l_y}{l_y - v_y} \Leftrightarrow p_x = \frac{l_y v_x - l_x v_y}{l_y - v_y}$$
- Z-coordinate is obtained in the same way. Y-coordinate is 0.
- These equations are converted into the projection matrix M. (A matrix such that $Mv = p$)

Planar Shadows – Projection Shadows

$$\bullet \quad M = \begin{pmatrix} l_y & -l_x & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -l_z & l_y & 0 \\ 0 & -1 & 0 & l_y \end{pmatrix} \text{ for the special case.}$$

- In the general case, the plane the shadow is cast is not $y = 0$, but instead $n \cdot x + d = 0$
- The ray which goes through v is intersected by the plane.
- The projected point $p = l - \frac{d + n \cdot l}{n \cdot (v - l)} (v - l)$

Planar Shadows – Projection Shadows

- Convert to projection matrix (7.4)

$$\begin{aligned}
 \bullet \text{ } M = & \begin{array}{cccc}
 n \cdot l + d - l_x n_x & -l_x n_y & -l_x n_z & -l_x d \\
 -l_y n_x & n \cdot l + d - l_y n_y & -l_y n_z & -l_y d \\
 -l_z n_x & -l_z n_y & n \cdot l + d - l_z n_z & -l_z d \\
 -n_x & -n_y & -n_z & n \cdot l
 \end{array}
 \end{aligned}$$

- Turns into the matrix from the special case if $y = 0$, because $n = (0, 1, 0)$ and $d = 0$.
- Simply applying the above matrix to the objects that should cast shadows on the plane and the rendering it with a dark color and no illumination produces shadows.

Planar Shadows

– Projection

Shadows

- Z-fighting : When the shadow is projected onto the plane, the shadow triangles and the ground plane exist at the same depth. Due to floating point errors, the GPU cannot determine which one is 'in front'.
 - Method 1 : Adding bias to the shadow
 - **Method 2** : Z-buffer manipulation through drawing order (**safer method**)
 - Z-buffer is a tool the GPU uses to keep track of how far away pixels are. Normally, it only draws a pixel if it is closer than what is already there.
1. Draw the ground.
 2. Disable Z-buffer. This tells the GPU to simply paint over whatever is already there without checking if the next object is deeper than the floor.
 3. Draw the shadow.
 4. Re-enable Z-buffer.

Planar Shadows – Projection Shadows

- If the ground plane has a limit, and not infinite, the projection matrix method causes 'floating' shadows.
- Solution – The stencil buffer
- The stencil buffer is an offscreen buffer used to record the locations of the rendered primitive. (pg. 24)
- Draw the receiver to the screen and to the stencil buffer. Then, with the z-buffer off, draw the projected triangles only where the receiver was drawn.

Planar Shadows – Projection Shadows

- Another approach – render the shadow triangles into a texture which is applied to the ground plane.
 - **Pros** : Allows penumbrae and shadows on curved surfaces.
 - **Cons** : The texture can become ‘magnified’ with a single texel covering multiple pixels.
(Shadow looks blocky or jagged.)

Planar Shadows – Projection Shadows

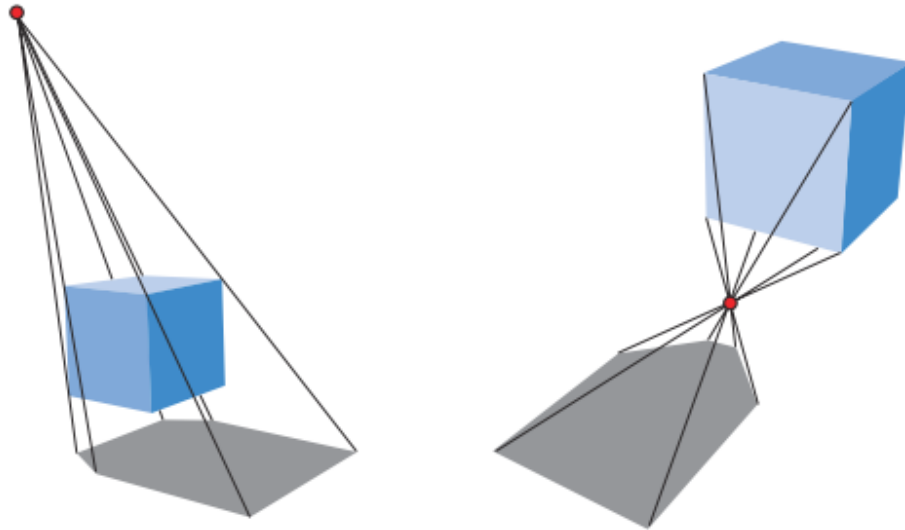


Figure 7.4. At the left, a correct shadow is shown, while in the figure on the right, an antishadow appears, since the light source is below the topmost vertex of the object.

- Antishadow – Generated if the light source is below the topmost point on the object since each vertex is projected through the point of the light source.
- An error will also occur if we project an object that is below the receiving plane.

Planar Shadows – Soft Shadows

- Heckbert and Herf's algorithm – Goal is to generate a texture on a ground plane that shows a soft shadow.
- The area light source is treated as **a collection of many punctual light sources**.
- A separate shadow is rendered for each punctual light source. The rendered shadows are accumulated and averaged.
- Where all shadows overlap, it is the **umbra**, and it is the **penumbra** where only some overlap.
- A frustum-based method – The idea is to treat the light as the viewer, and the ground plane forms the far clipping plane of the frustum.

Planar Shadows – Soft Shadows

- Problem with the sampled area-light method – For n shadow passes, only $n + 1$ distinct shades can be generated.
- Larger number of passes $\Leftrightarrow$ Accurate result but excessive cost
- More efficient methods:
 - **Convolution.**
 - Blurring a hard shadow.
 - Haines' algorithm – starts with a projected hard shadow and then renders the silhouette edges to create plausible penumbrae.

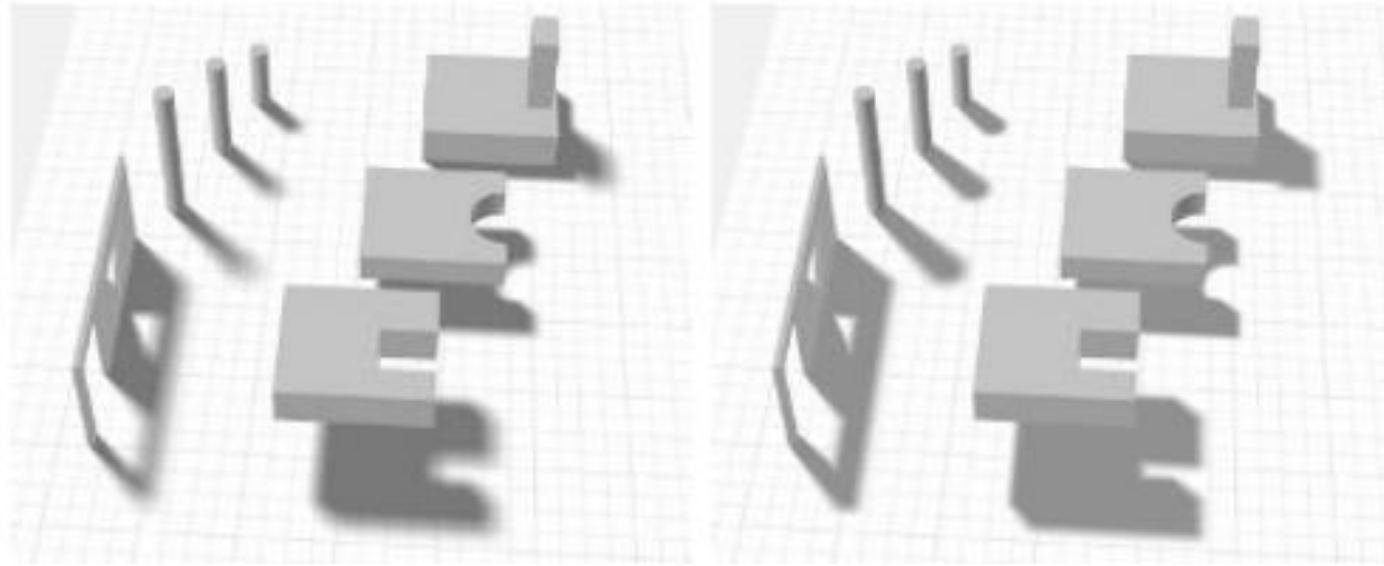


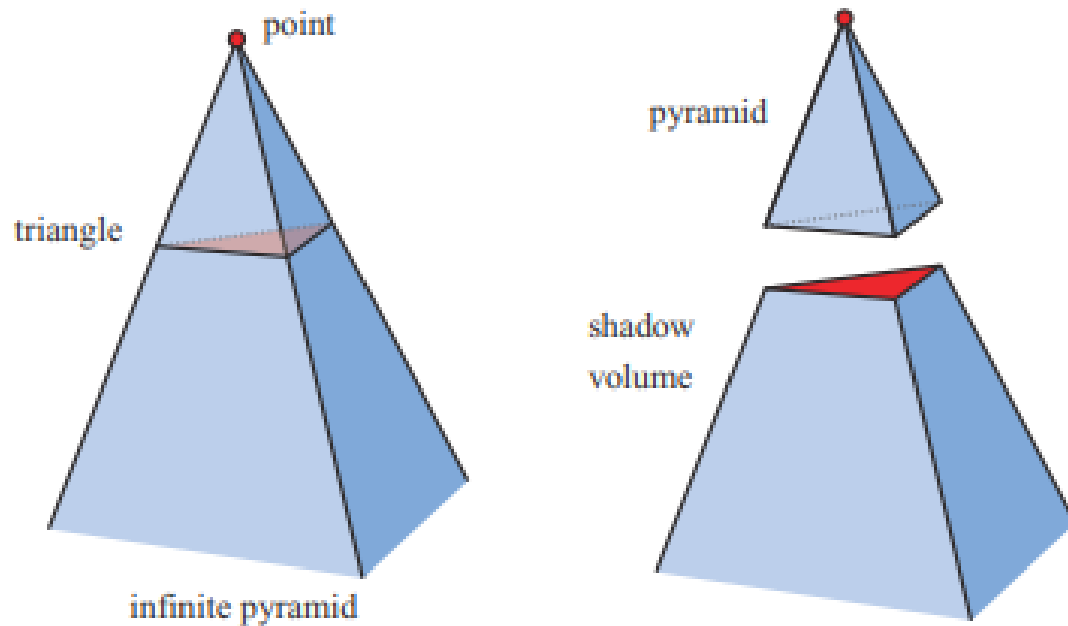
Figure 7.5. On the left, a rendering using Heckbert and Herf's method, using 256 passes. On the right, Haines' method in one pass. The umbrae are too large with Haines' method, which is particularly noticeable around the doorway and window.

Planar Shadows – Soft Shadows

Shadows on Curved Surfaces – Texture Projection

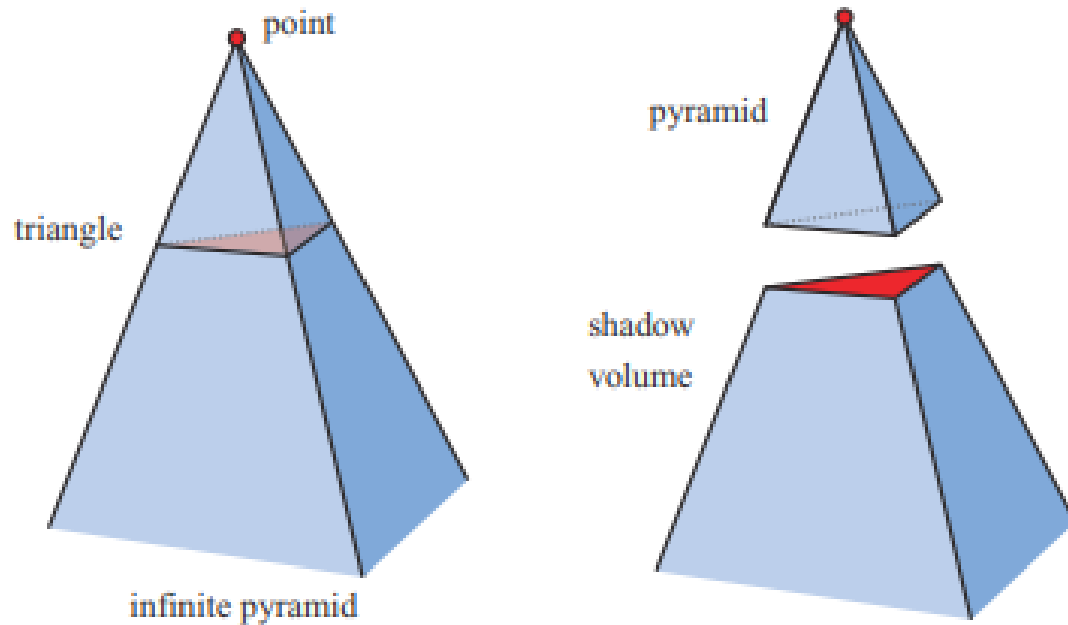
- What the light cannot 'see' is shadow.
- Occluder is rendered in black from the light's viewpoint into an otherwise white texture.
- The texture is then projected onto the surfaces.
- Each vertex on the receivers has a (u, v) texture coordinate computed for it and has the texture applied to it.
- **Drawbacks :**
 - Application must identify which objects are occluders and which are receivers.
 - Occluding objects cannot shadow themselves.

Shadow Volumes



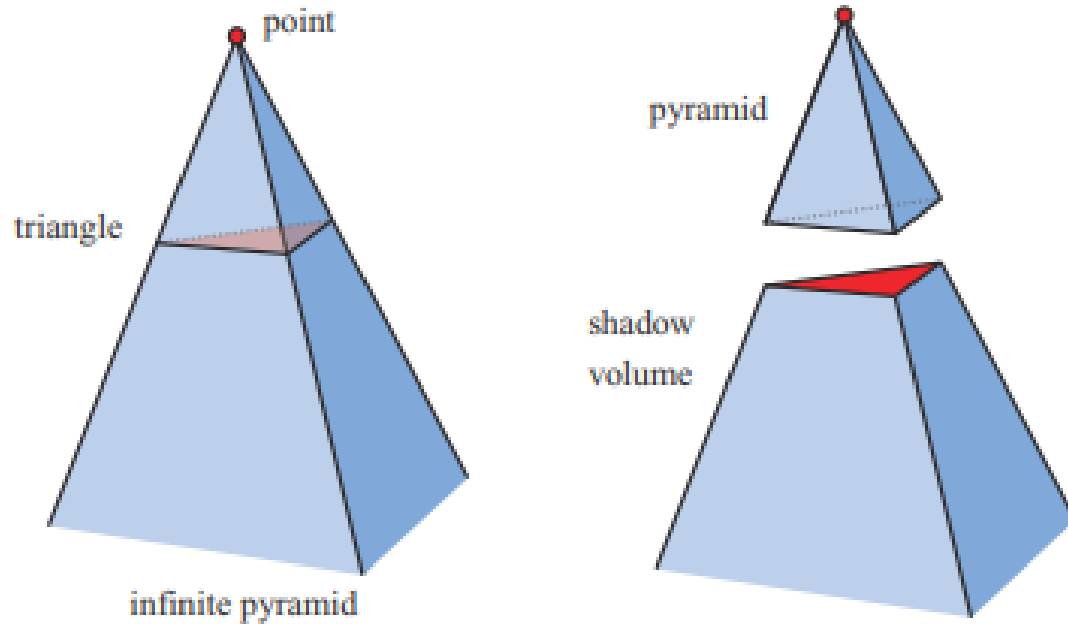
- Not image based thus avoiding sampling problems to produce correct sharp shadows everywhere. (Sometimes a disadvantage)
- Imagine following a ray from the eye through a pixel until the ray hits the object to be displayed.
- While the ray is on its way to the object, increment a counter each time it crosses a face of the shadow volume that is frontfacing.

Shadow Volumes



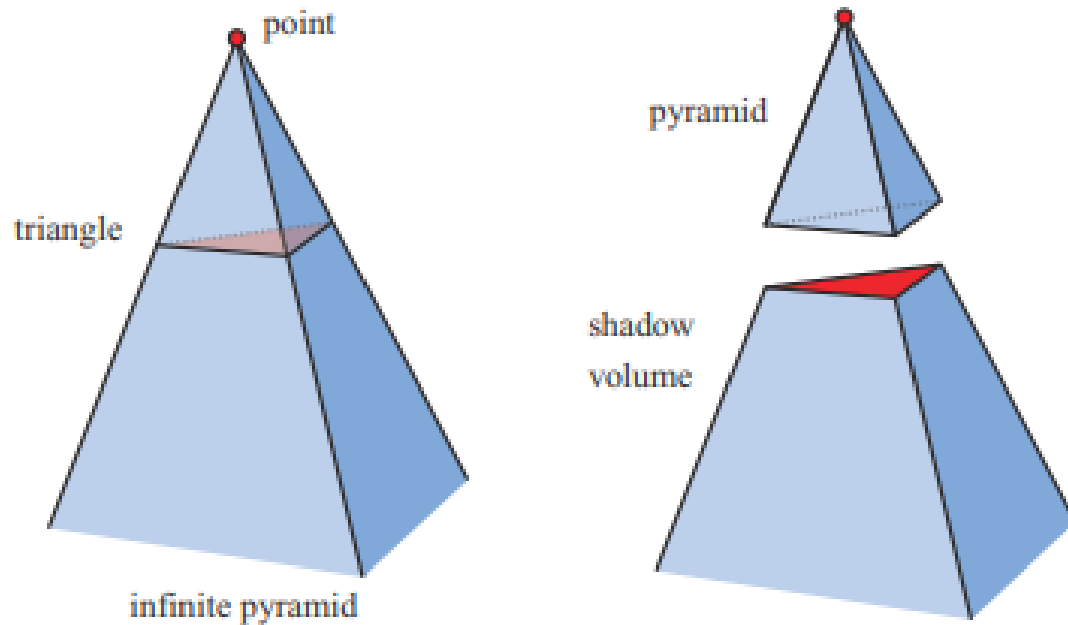
- Similarly, decrement the counter each time the ray crosses a backfacing face of the truncated pyramid. (Ray is going out of shadow)
- Increment and decrement counter until ray hits the object to be displayed at the pixel. If counter is greater than zero, the pixel is in shadow.

Shadow Volumes



- Performing the procedure with actual rays is time consuming.
- Solution: Stencil buffer

Shadow Volumes



1. The stencil buffer is cleared.
2. Whole scene is drawn into the framebuffer with only the color of the unlit material used.
3. Z-buffer updates and writing to the color buffer are turned off. Frontfacing triangles of shadow volumes are drawn. Stencil operation is set to increment the values in the stencil buffer wherever a triangle is drawn.
4. Another pass of the stencil buffer is carried out for the backfacing triangles. (Decrement counter. Counter is only updated when the pixels of the rendered shadow volume face are visible.)
5. Whole scene is rendered again with only the active materials that are affected by the light.

Shadow Volumes

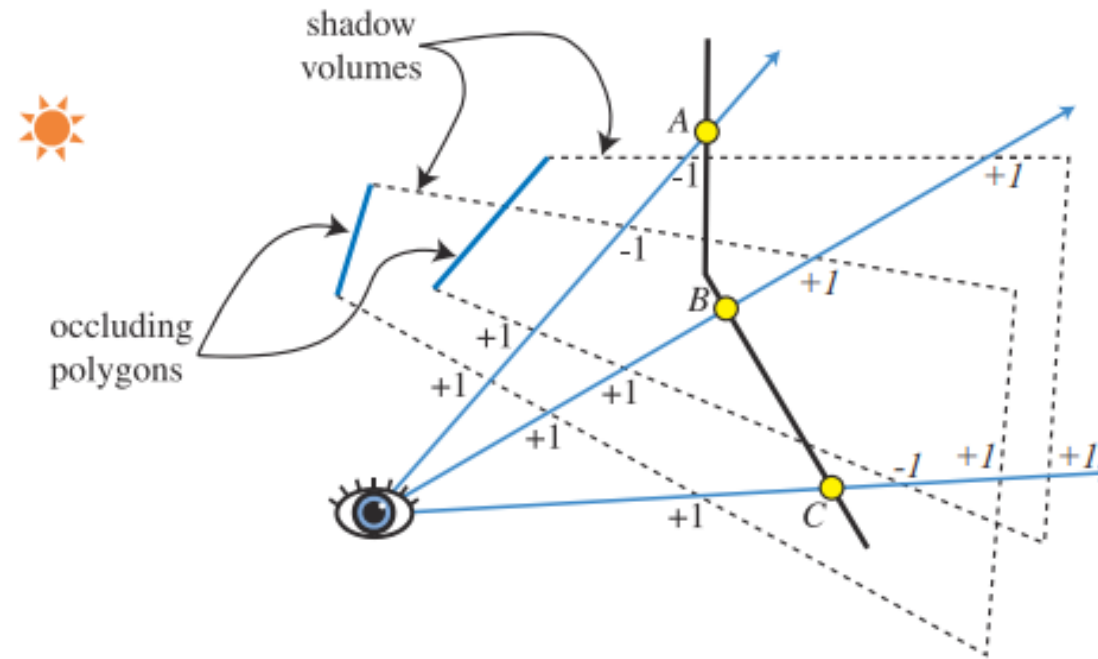


Figure 7.8. A two-dimensional side view of counting shadow-volume crossings using two different counting methods. In *z-pass* volume counting, the count is incremented as a ray passes through a frontfacing triangle of a shadow volume and decremented on leaving through a backfacing triangle. So, at point *A*, the ray enters two shadow volumes for *+2*, then leaves two volumes, leaving a net count of zero, so the point is in light. In *z-fail* volume counting, the count starts beyond the surface (these counts are shown in *italics*). For the ray at point *B*, the *z-pass* method gives a *+2* count by passing through two frontfacing triangles, and the *z-fail* gives the same count by passing through two backfacing triangles. Point *C* shows how *z-fail* shadow volumes must be capped. The ray starting from point *C* first hits a frontfacing triangle, giving *-1*. It then exits two shadow volumes (through their endcaps, necessary for this method to work properly), giving a net count of *+1*. The count is not zero, so the point is in shadow. Both methods always give the same count results for all points on the viewed surfaces.

Shadow Volumes



Figure 7.9. Shadow volumes. On the left, a character casts a shadow. On the right, the extruded triangles of the model are shown. (*Images from Microsoft SDK [1208] sample "ShadowVolume."*)

Shadow Volumes

- Drawback : Creating quadrilaterals for every triangle creates a huge amount of overdraw.
- Solution : Draw only quadrilaterals along the silhouette edges of the object.

Shadow Volumes

- Another Drawback : **Extreme variability in computation**
- Makes it unusable in interactive applications where a consistent frame rate is important (such as online games).

Shadow Maps

- In 1978, Williams proposed that a common z-buffer based renderer could be used to generate shadows quickly on arbitrary objects.
- Render the scene using the z-buffer, from the position of the light source that is to cast shadows. Each pixel in the z-buffer now contains the z-depth of the object closest to the light source. We call the entire contents of the z-buffer the **shadow map**.
- The scene is then rendered a second time with respect to the viewer. As each primitive is rendered, its location at each pixel is compared to the shadow map.
- If rendered point is...
 - Farther than the corresponding value in the shadow map, that point is in shadow.
 - Closer than the corresponding value in the shadow map, that point is not in shadow.

Shadow Maps

- Solves the variability problem with shadow volumes.
- Cost is linear with the number of rendered primitives, and access time is constant.

Shadow Maps

- When a single z-buffer is generated, the light can only see in one direction just like a camera.
- For **directional light**, the view is set to encompass all objects casting shadows into the viewing volume that the eye sees. (Orthographic projection) The light's view needs to be made wide and high enough in x and y to view this set of objects.
- For **spotlight**, it naturally defines its own frustum based on its cone angle (perspective projection). Anything outside the cone isn't illuminated so the shadow map only needs to cover that specific area.

Shadow Maps

- **Problem case** : Local light source is inside a scene and is surrounded by shadow-casters.
- Light shines in all directions which cannot be covered by a single frustum.
- **Solution** : Use a six-view cube. (A cube map texture)
- Also called omnidirectional shadow maps.
- **Challenge** : Avoiding along the seams where two separate maps meet.
- Research has been conducted by many researchers, but their details are not introduced in this text. (King, Newhall, Gerasimov, Forsyth, Crytek)

Shadow Maps

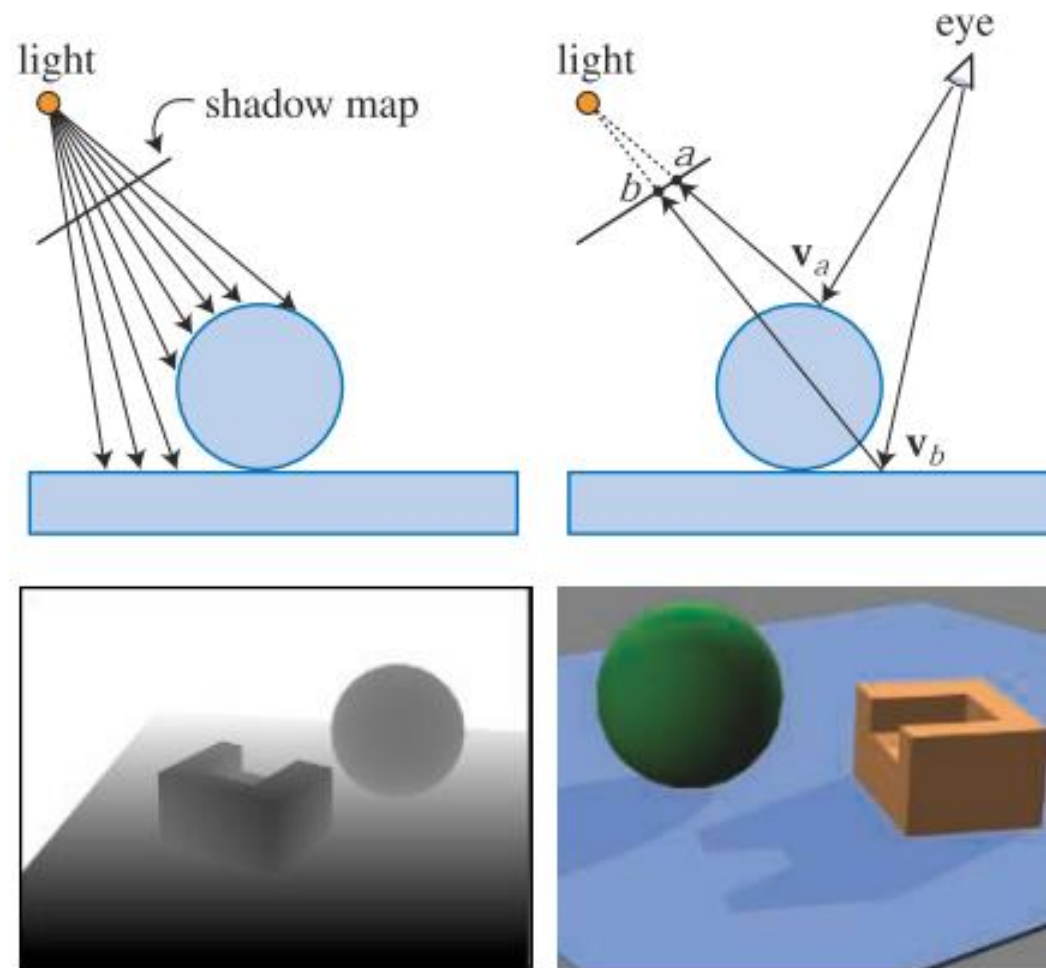


Figure 7.10. Shadow mapping. On the top left, a shadow map is formed by storing the depths to the surfaces in view. On the top right, the eye is shown looking at two locations. The sphere is seen at point v_a , and this point is found to be located at texel a on the shadow map. The depth stored there is not (much) less than point v_a is from the light, so the point is illuminated. The rectangle hit at point v_b is (much) farther away from the light than the depth stored at texel b , and so is in shadow. On the bottom left is the view of a scene from the light's perspective, with white being farther away. On the bottom right is the scene rendered with this shadow map.

Shadow Maps

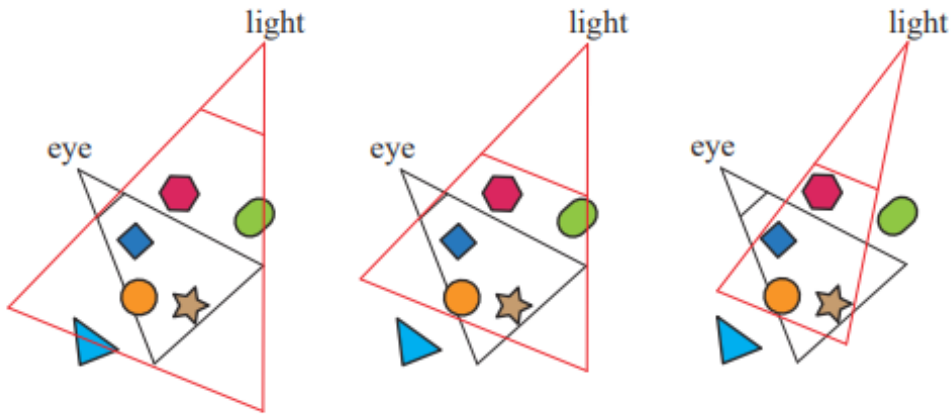


Figure 7.11. On the left, the light's view encompasses the eye's frustum. In the middle, the light's far plane is pulled in to include only visible receivers, so culling the triangle as a caster; the near plane is also adjusted. On the right, the light's frustum sides are made to bound the visible receivers, culling the green capsule.

- Not all objects need to be rendered into the shadow map just the shadow casters.
- The light's view frustum can be augmented or tightened to disregard some shadow casters.
- Rendering only relevant objects not only can save time rendering but can also reduce the size required for the light's frustum thus increasing quality.
- Effective precision of the z-buffer increases if...
 - The light frustum's near plan is as far away from the light as possible.
 - The far plane is as close to the light as possible.

Shadow Maps

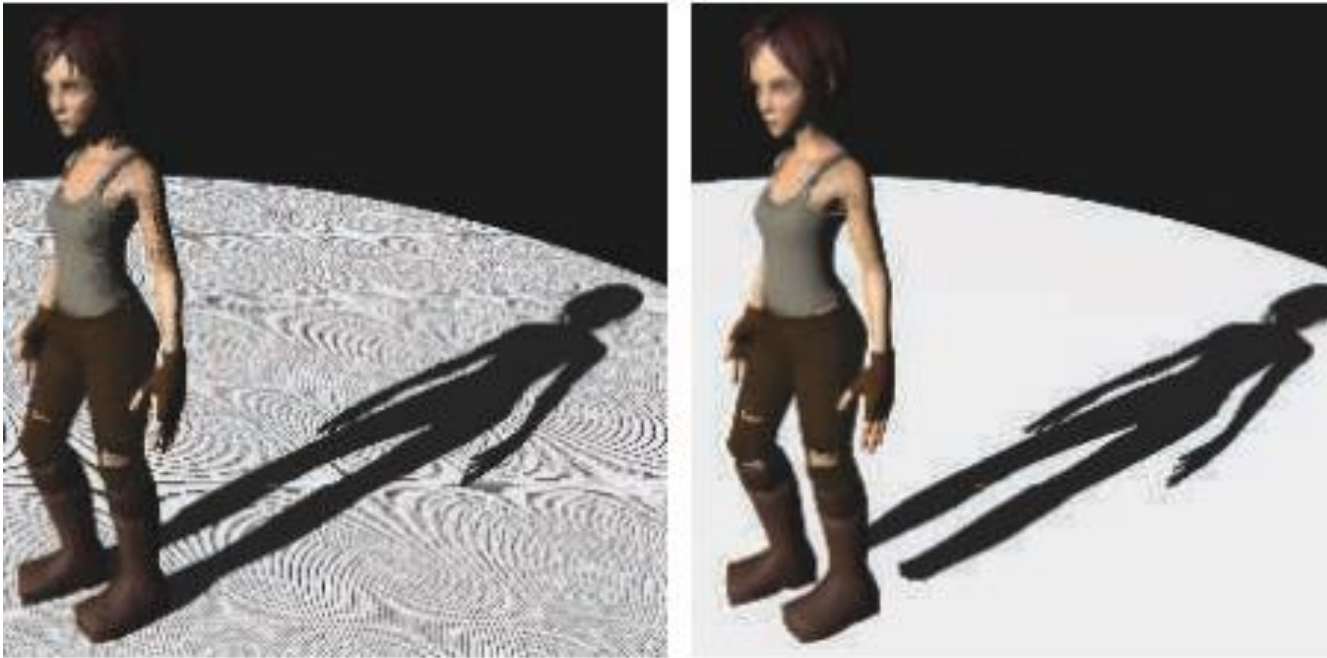


Figure 7.12. Shadow-mapping bias artifacts. On the left, the bias is too low, so self-shadowing occurs. On the right, a high bias causes the shoes to not cast contact shadows. The shadow-map resolution is also too low, giving the shadow a blocky appearance. (Images generated using a shadow denoiser by Christoph Peters.)

- A **disadvantage** of shadow mapping – The quality of the shadows depends on the resolution of the shadow map and on the numerical precision of the z-buffer.
- Also susceptible to aliasing problems.
- **Self-shadow aliasing** (also known as shadow acne) when a triangle is incorrectly considered to shadow itself.
 - Numerical limits of precision.
 - The value of a point sample is being used to represent an area's depth. (Samples generated for the light are almost never at the same locations as the screen samples)

Shadow Maps

- A remedy (but not a fix) is to introduce a bias factor.
- A small bias is subtracted from the receiver's distance when checking the distance found in the shadow map with the distance of the location being tested.
- **Constant** bias can fail when the receiver is not mostly facing the light.
- More effective – Use a bias that is proportional to the angle of the receiver to the light. (Slope scale bias) Often clamped to avoid extremely high tangent values.
- Both methods are used side by side to avoid possible precision errors.

Shadow Maps

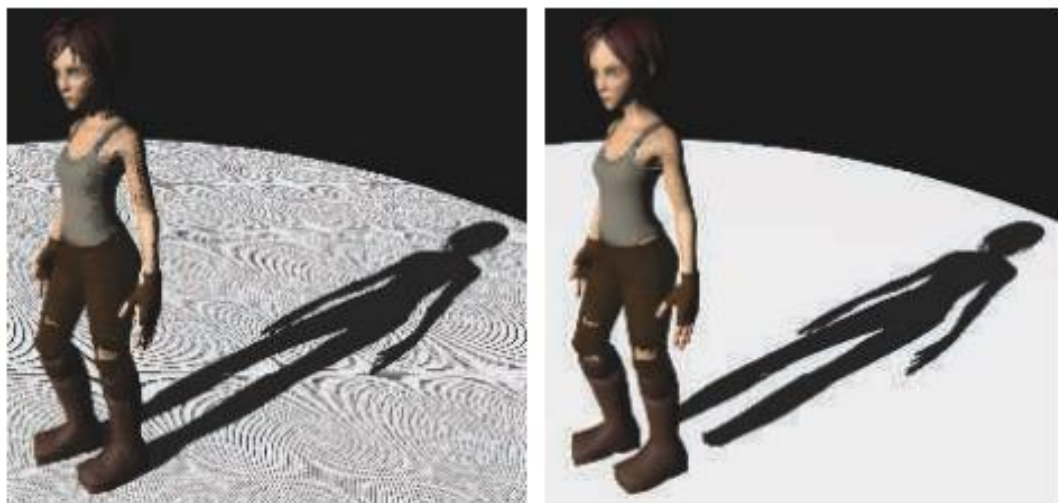


Figure 7.12. Shadow-mapping bias artifacts. On the left, the bias is too low, so self-shadowing occurs. On the right, a high bias causes the shoes to not cast contact shadows. The shadow-map resolution is also too low, giving the shadow a blocky appearance. (Images generated using a shadow demo by Christoph Peters.)

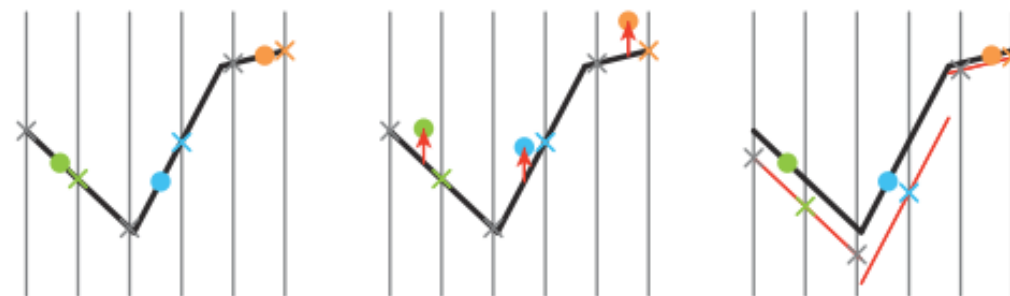


Figure 7.13. Shadow bias. The surfaces are rendered into a shadow map for an overhead light, with the vertical lines representing shadow-map pixel centers. Occluder depths are recorded at the $\times$ locations. We want to know if the surface is lit at the three samples shown as dots. The closest shadow-map depth value for each is shown with the same color $\times$. On the left, if no bias is added, the blue and orange samples will be incorrectly determined to be in shadow, since they are farther from the light than their corresponding shadow-map depths. In the middle, a constant depth bias is subtracted from each sample, placing each closer to the light. The blue sample is still considered in shadow because it is not closer to the light than the shadow-map depth it is tested against. On the right, the shadow map is formed by moving each polygon away from the light proportional to its slope. All sample depths are now closer than their shadow-map depths, so all are lit.

Shadow Maps

- Another solution to self-shadowing – Render only the backfaces to the shadow map.
- Second-depth shadow mapping.
- **Problem cases** : Object is two-sided, thin, or in contact with one another.
- Sousa et al. – Frontfaces work well for sun shadows and backfaces for interior lights.

Shadow Map - Resolution Enhancement

- Goal : One shadow-map texel covers about one image pixel
- Ideal scenario – light source located at the same position as the eye
- Change in light direction causes change in texel to pixel ratio causing artifacts.
- Perspective aliasing – Large number of foreground pixels are associated with each texel of the shadow map
- Projective aliasing – Single shadow-map texel covers many pixels because a surface is nearly edge-on to the light but faces the viewer
- Simple fix: Increase shadow-map resolution at the cost of additional memory and processing

Resolution Enhancement

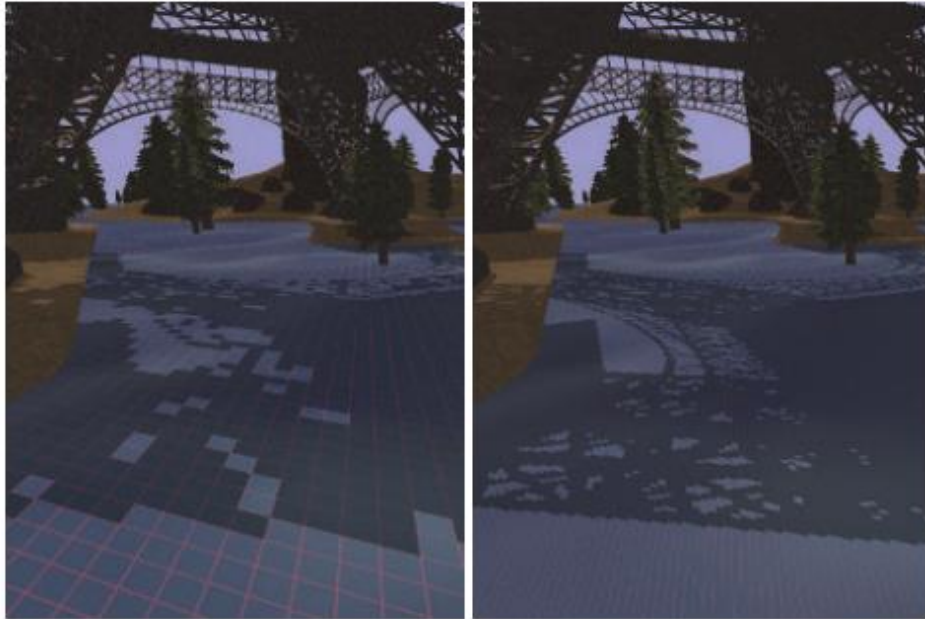


Figure 7.15. The image to the left is created using standard shadow mapping; the image to the right using LiSPSM. The projections of each shadow map's texels are shown. The two shadow maps have the same resolution, the difference being that LiSPSM reforms the light's matrices to provide a higher sampling rate nearer the viewer. (Images courtesy of Daniel Scherzer, Vienna University of Technology.)



Figure 7.16. On the left the light is nearly overhead. The edge of the shadow is a bit ragged due to a low resolution compared to the eye's view. On the right the light is near the horizon, so each shadow texel covers considerably more screen area horizontally and so gives a more jagged edge. (Images generated by TheRealMJP's "Shadows" program on Github.)

Resolution Enhancement

- The window defining the frustum can be shifted, skewed, or rotated on the view plane.
- The view direction only defines the view plane and not the pixels sampled.
- 22 degrees of freedom in mapping the light's view to the eye's.
- PSM, TSM, LiSPSM are perspective warping methods that build on the above solution attempting to better match the light's sampling rates to the eye's.
- Matrix warping techniques.

Resolution Enhancement

- One light situation where matrix-warping techniques fail to help is when a light is in front of the camera and pointing at it.
- Dueling frusta
- More shadow-map samples are needed nearer the eye
- Simple idea: Generate several shadow maps at different resolutions for a given view.
- Use high resolution maps for nearby objects and low resolution for far away objects.

Resolution Enhancement

- Z-partitioning : Deciding the range of z-depths to split the maps
- Logarithmic partitioning

Resolution Enhancement

- $r = \sqrt[c]{\frac{f}{n}}$
- n and f : Near and far clipping planes of the entire scene
- c : The number of shadow maps to use
- r : The ratio that determines the boundary of each map
- n=1, f=1000, c=3, r=10
- 1m to 10m, 10m to 100m, 100m to 1000m

Resolution Enhancement

- Near depth is very important.
- Lower near depth causes the shadow map generated to cover a larger area, lowering its precision.
- Setting the near plane too far causes objects to be clipped by the plane.

Resolution Enhancement

- Sample Distribution Shadow Maps
- Uses z-depth values from the previous frame to determine a better partitioning
- First method – look through the z-depths for the minimum and maximum values and use these to set the near and far planes (uses reduce operation on the GPU).
 - Quick and general and gives good enough results
- Second method – make a histogram that records the distribution of the z-depths along the range.
 - Gaps may represent where there are no objects at all.

Further Enhancements

- If nothing changes within a shadow map's frustum, no need for recomputation.
- Remove tiny enough occluders from consideration.
- More distant shadow maps may be updated less frequently.

Percentage-Closer Filtering

- Provides pseudo-soft shadows.
- Instead of a single sample being taken off the shadow map, the four nearest samples are retrieved.
- The surface's depth is compared separately to the four texel depths.
- The results are used to calculate how much the light actually contributes to the surface location, causing an artificially soft shadow.

Percentage-Closer Filtering

- Goal of PCF – Attempts to approximate a soft shadow for punctual light
- Instead of finding the light's visible area from a surface location, find the visibility of the punctual light from a set of surface locations¹
- Find the percentage of the samples taken that are visible to the light.

Percentage-Closer Filtering

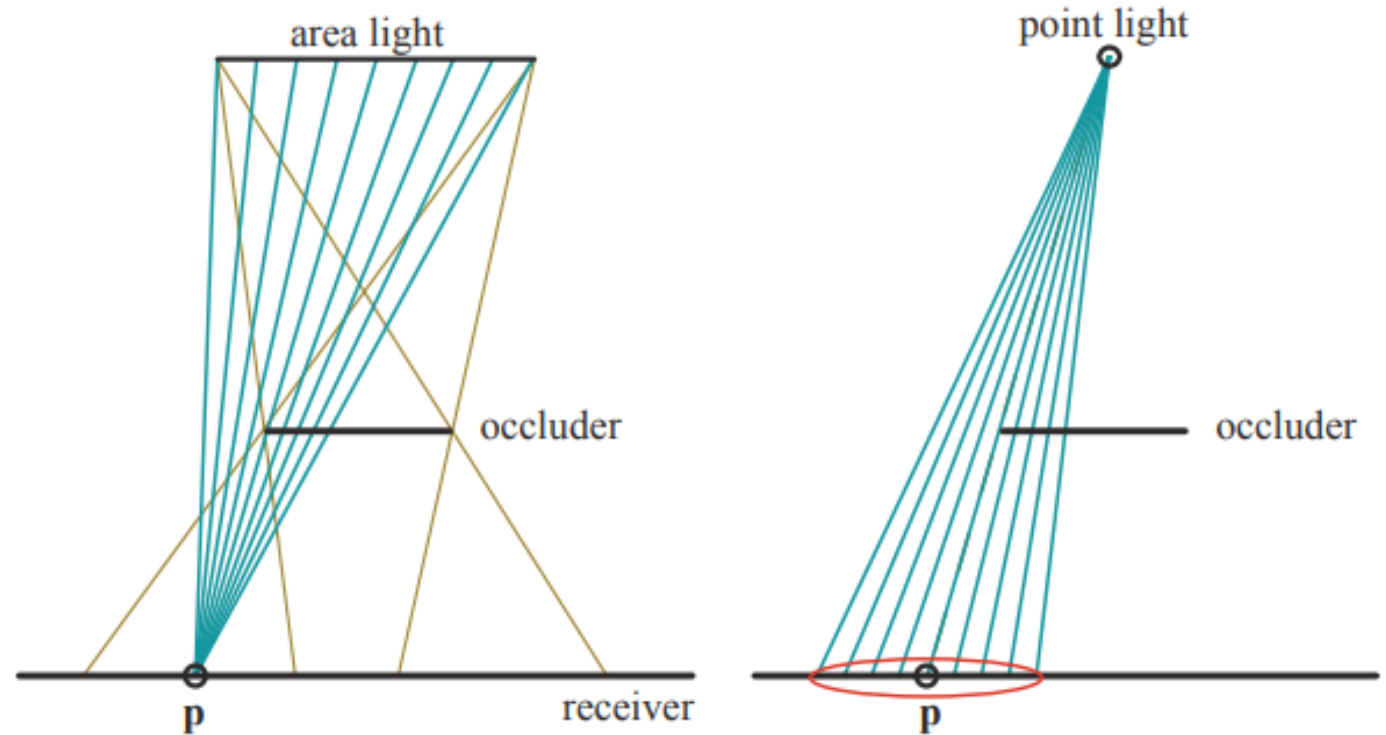


Figure 7.22. On the left, the brown lines from the area light source show where penumbrae are formed. For a single point p on the receiver, the amount of illumination received could be computed by testing a set of points on the area light's surface and finding which are not blocked by any occluders. On the right, a point light does not cast a penumbra. PCF approximates the effect of an area light by reversing the process: At a given location, it samples over a comparable area on the shadow map to derive a percentage of how many samples are illuminated. The red ellipse shows the area sampled on the shadow map. Ideally, the width of this disk is proportional to the distance between the receiver and occluder.

Percentage-Closer Filtering

Simply using four neighboring samples in a regular grid pattern can create noticeable artifacts.

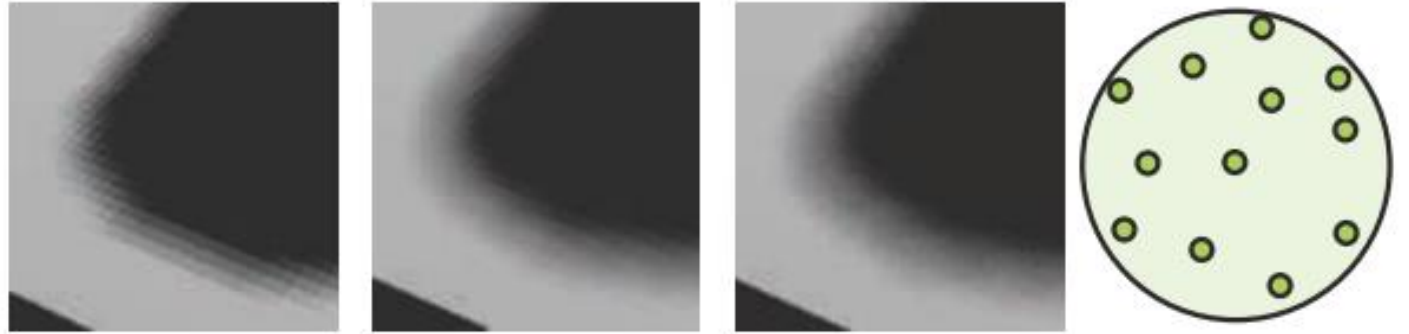


Figure 7.23. The far left shows PCF sampling in a 4×4 grid pattern, using nearest neighbor sampling. The far right shows a 12-tap Poisson sampling pattern on a disk. Using this pattern to sample the shadow map gives the improved result in the middle left, though artifacts are still visible. In the middle right, the sampling pattern is rotated randomly around its center from pixel to pixel. The structured shadow artifacts turn into (much less objectionable) noise. (Images courtesy of John Isidoro, ATI Research, Inc.)

Percentage-Closer Filtering

- One problem : The sampling area's width remains constant.
- Shadows appear uniformly soft.
- Appears incorrect where there is ground contact between the occluder and receiver.



Figure 7.25. Percentage-closer filtering and percentage-closer soft shadows. On the left, hard shadows with a little PCF filtering. In the middle, constant width soft shadows. On the right, variable-width soft shadows with proper hardness where objects are in contact with the ground. (Images courtesy of NVIDIA Corporation.)

Percentage-Closer Soft Shadows

- Published by Fernando in 2005.
- Attempts a solution by searching the nearby area on the shadow map to find all possible occluders.
- Average distance of occluders from the location is used to determine the sample area width.
- If no occluders, location is fully lit!
- If location is fully occluded, hard shadow.

Percentage-Closer Soft Shadows

- Drawback: Requires a fair-size area sample.
- Sikachev et al. – A faster implementation of PCSS using features introduced by AMD called ‘contact hardening shadows’.
 - Address another problem with PCSS which is that the penumbra’s size is affected by the shadow map’s resolution.
 - First generates mipmaps (a set of bitmap images to represent a texture) of the shadow map, then chooses the mip level closest to a user-defined world-space kernel size.
 - 8 x 8 area sampled to find average blocker depth.
 - Once penumbra estimate found, higher-resolution mipmap are used for the sharp area of the shadow.

Percentage-Closer Soft Shadows

- Major assumption – Average blocker is a reasonable estimate of the penumbra size.
- Counterexample : A street lamp and a distant mountain that partially occlude the same surface.
 - There is the need to determine how much of the area light source is visibly from a receiver location.
- Backprojection using the GPU – Treat each receiver's location as a viewpoint and the area light source as part of a view plane, and to project occluders onto this plane.
 - Excellent results but high cost
 - Not adopted in interactive applications yet

Filtered Shadow Maps

- Variance Shadow Map – Store both the depth and depth squared in another map
- Depth map is sampled first at the receiver's location to return an average depth of the closest light occluder. -> M_1 = first moment
- If $M_1 > \text{depth on the shadow receiver } t$, receiver is considered fully in light.
- Otherwise,
$$p_{\max}(t) = \frac{\sigma^2}{\sigma^2 + (t - M_1)^2},$$
- $p_{\max}$ is the maximum percentage of samples in light, σ^2 is the variance, t is the receiver depth.
- Average of depth squared shadow map's sample is called M_2 /second moment.
- $\sigma^2 = M_2 - M_1^2$

Filtered Shadow Maps

- Goal – Attempts to estimate how much of the distribution of occluders at the surface location is beyond the surface's distance from the light.
- Intuition – The variance over an area increases at shadow edges. Greater the difference in depths, greater the variance.
- Moving into the penumbra, average occluder depth gets closer to the light.

Filtered Shadow Maps

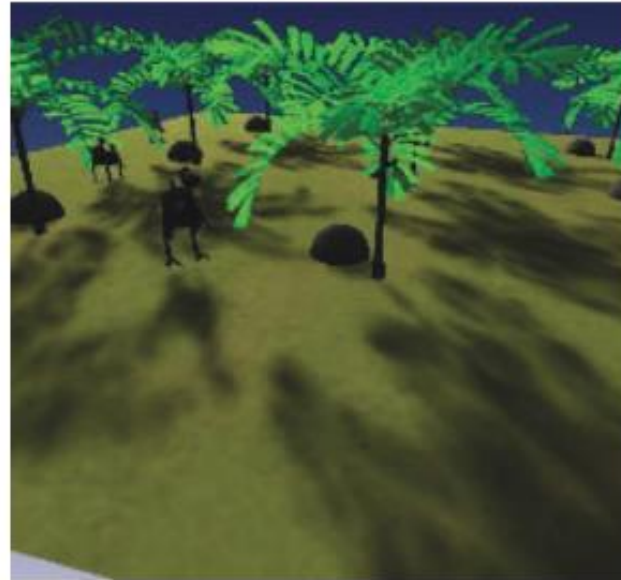
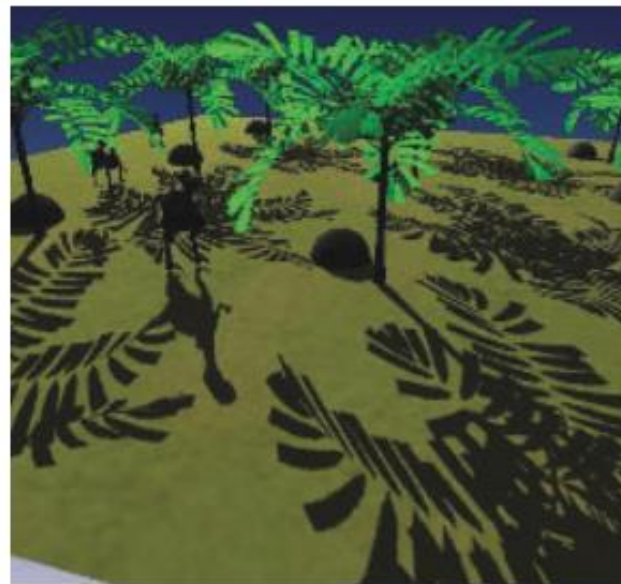
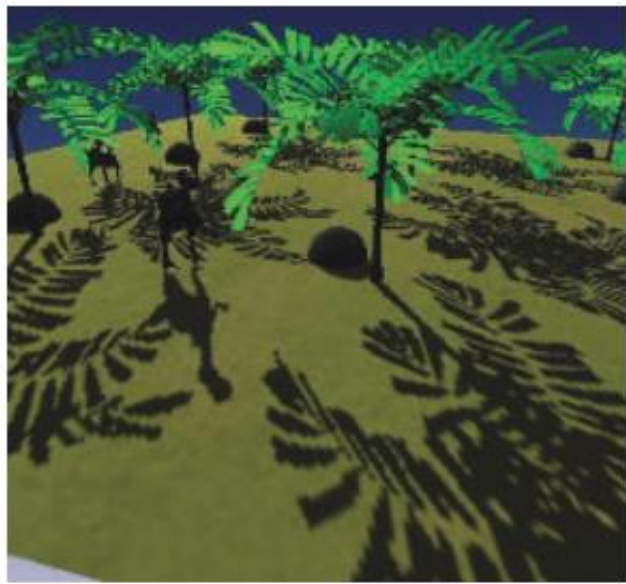
- VSM vs PCF – PCF needs more samples and thus more time. VSM works with a single high-quality sample.
- Drawback – Along penumbrae areas when two or more occluders cover a receiver and one occluder is close to the receiver.
 - The closest occluder throws the equation off by only partially hiding the light (light bleeding – false lighting)
 - Can overcome by taking more samples over a smaller area but turns VSM into a form of PCF.
 - Approach by Lauritzen – Treat low percentages as fully shadowed and remap the rest of the percentage range from 0 to 100%.
 - Darkens light bleeds but narrower penumbrae overall.

Filtered Shadow Maps

- Convolution Shadow Map – Encode the shadow depth in a Fourier expansion (?).
- Drawback : Several terms need to be computed and accessed, considerably increasing both execution and storage costs.

Filtered Shadow Maps

- Exponential Variance Shadow Map – Save the exponential of the depth along with its second moment into two buffers.
- Exponential more closely approximates the step function that a shadow map performs. Thus, reduced bleeding artifacts.
- Limitation : Second moment values can become extremely large causing floating-point errors.
- Still of a lot of interest because
 - Improved quality over VSM
 - Lower storage and better performance compared to convolution.



Filtered Shadow Maps

Figure 7.26. In the upper left, standard shadow mapping. Upper right, perspective shadow mapping, increasing the density of shadow-map texel density near the viewer. Lower left, percentage-closer soft shadows, softening the shadows as the occluder's distance from the receiver increases. Lower right, variance shadow mapping with a constant soft shadow width, each pixel shaded with a single variance map sample. (Images courtesy of Nico Hempe, Yvonne Jung, and Johannes Behr.)

Filtered Shadow Maps

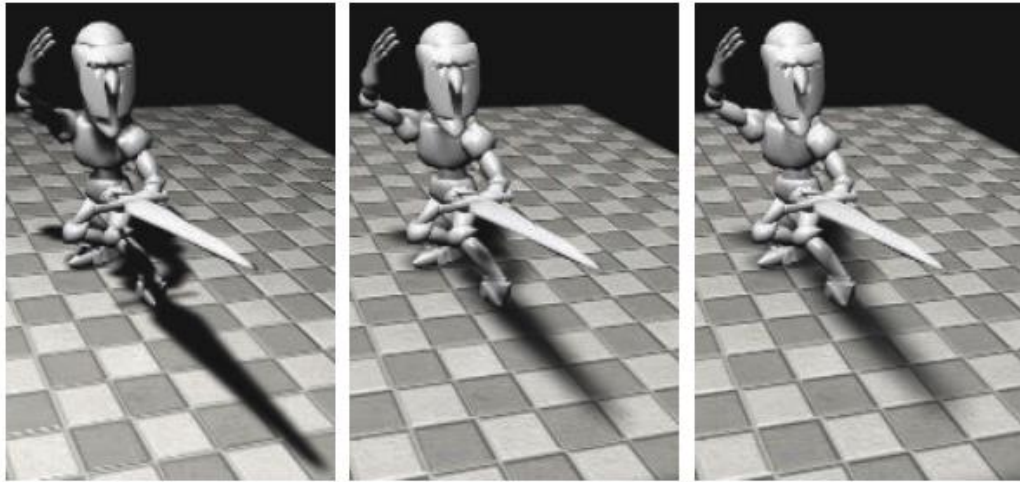


Figure 7.27. Variance shadow mapping, where the distance to the light source increases from left to right. (Images from the NVIDIA SDK 10 [1300] samples, courtesy of NVIDIA Corporation.)

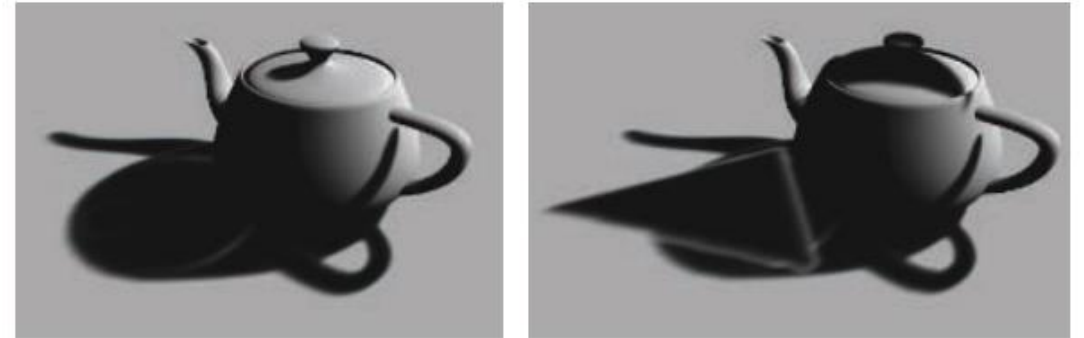


Figure 7.28. On the left, variance shadow mapping applied to a teapot. On the right, a triangle (not shown) casts a shadow on the teapot, causing objectionable artifacts in the shadow on the ground. (Images courtesy of Marco Salvi.)

Volumetric Shadow Techniques

- Imagine transparent objects such as stained glass. The standard shadow map simply tells you that there is an object and you get a black shadow.
- Secondary shadow map that renders the transparent objects' color and opacity.
- When GPU renders the receiver, first checks the opaque shadow map. If the object is not blocked by an opaque occluder, then the transparency map is checked.
- If there is for example, green glass, the ground is tinted green.

Volumetric Shadow Techniques

- Semitransparent objects such as hair and clouds – self shadowing
- Lokovic and Veach – Deep shadow maps
- Each shadow map texel stores a function of how light drops off with depth.
- Typically approximated by a series of samples at different depths with each sample having an opacity value.
- The two samples that bracket a given position's depth determine the shadow's effect.
- Challenge: Generate and evaluate the functions efficiently.

Volumetric Shadow Techniques

- Kim and Neumann – GPU based method
- Opacity Shadow Maps
- Stores just the opacities generated at a fixed set of depths.
- Nguyen and Donnelly give an updated approach. Yuksel and Keyser improve efficiency and quality.
- Depth slices are still parallel and uniform.

Volumetric Shadow Techniques

- Salvi et al – Adaptive Volumetric Shadow Maps
- Avoid having to rely on fixed slice setups.
- Each shadow map texel stores both the opacities and layer depths.
- Pixel shader operations used to lossily compress data stream (surface opacities)
- Similar to deep shadow maps, but compression step is made efficient using shaders.

Volumetric Shadow Techniques



Figure 7.29. Hair and smoke rendering with adaptive volumetric shadow maps [1531]. (Reprinted by permission of Marco Salvi and Intel Corporation, copyright Intel Corporation, 2010.)

Irregular Z-buffer Shadows

- Reasons shadow-map approaches are popular
 - Predictable and scalable costs
 - Map nicely onto the GPU
- Problem: Light's view of the world cannot be mapped one-to-one with what the eye sees
- Aliasing problems when the light samples a surface less frequently than the eye

Irregular Z-buffer Shadows

- Shadow volumes provide an exact, analytical solution, as the light's interactions with surfaces result in sets of triangles defining whether any given location is lit or in shadow.
- Drawback : Unpredictable cost when implemented on the GPU

Irregular Z-buffer Shadows

- Another analytical method : Ray tracing
- A ray is shot from the receiver location to the light. If any object is found that blocks the ray, receiver is in shadow.
- Fast ray tracer – Dedicated to generating and using hierarchial data structures to minimize the number of object tests needed per ray.
- Continuing area of research

Irregular Z-buffer Shadows

- Another approach – Use the GPU's rasterization hardware with not just z-depths but additional information about the edges of the occluders in each grid cell of the light.
- Example, each shadow map texel stores a list of triangles that overlap the grid cell.
- Can be generated by conservative rasterization – Triangle generates a fragment if any part of the triangle overlaps a pixel not just the center.
- Problem : Large amount of data per texel. Needs to be limited.

Irregular Z-buffer Shadows

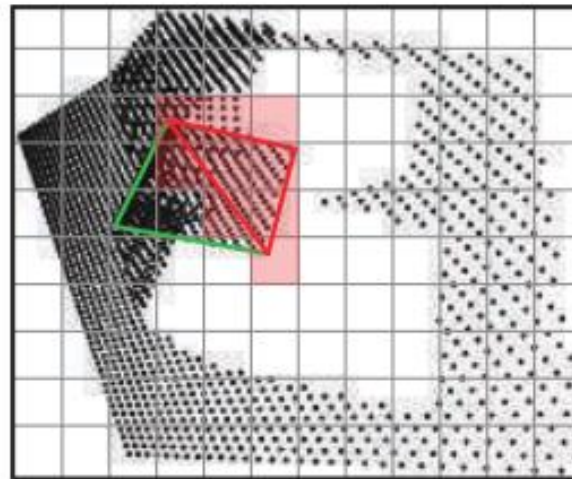
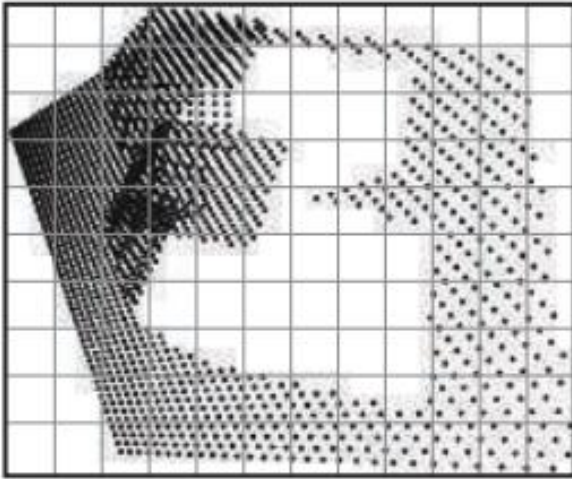
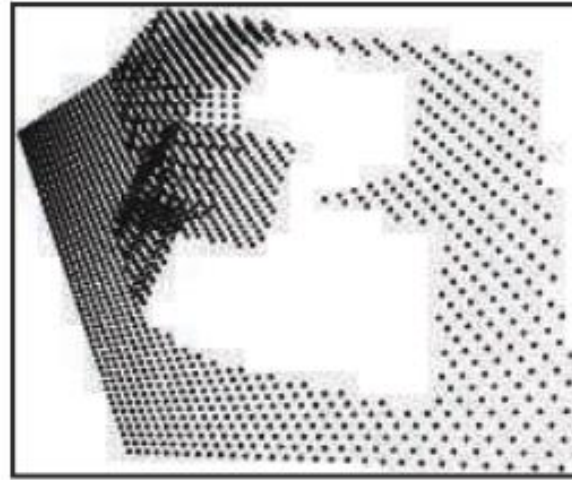
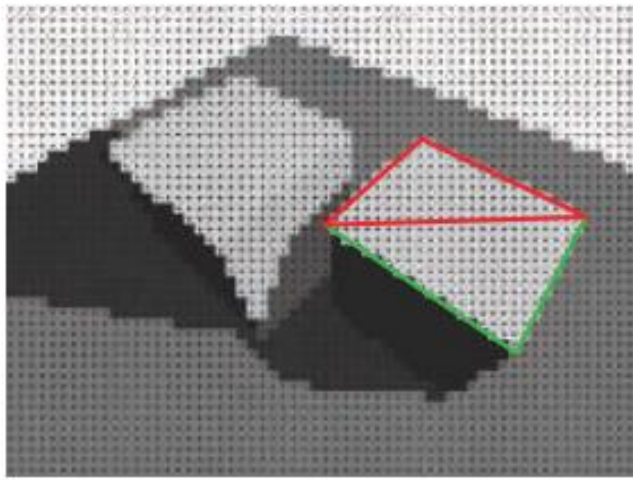
- Alternative to storing triangles – storing receiver locations and testing triangles against each.
- Johnson et al, Aila and Laine - Irregular z-buffer
- Misleading name – Z-buffer itself has a normal shape
 - Contents are irregular as each shadow map texel will have one or more receiver locations stored in it (sometimes none at all)

Irregular Z-buffer Shadows

- Sintorn et al, Wyman et al – Multi pass algorithm creates IZB and tests its contents for visibility from the light.
- First, the scene is rendered from the eye to find the z-depths of the surfaces seen from the eye.
- The points are then transformed to the light's view of the scene and tight bounds are formed from this set for the light's frustum.
- The points are then deposited in the light's IZB, each placed into a list at its corresponding texel.
- Empty list – A volume of space that the light views but has no surfaces seen by the eye.
- Conservative rasterization to determine whether any points are in shadow. Ensures that even partially overlapping points are tested.

Irregular Z-buffer Shadows

- Visibility testing in the pixel shader. Like ray tracing.
- Ray is generated from image point's location to the light. If a point is inside the triangle and more distant than the triangle's plane, it is hidden.
- Once all occluders are rasterized, light visibility results are used to shade the surface.
- Also called frustum tracing.
- Careful coding is critical to make this work well with the GPU. Huge changes in performance from changes in code.



Irregular Z-buffer Shadows

Figure 7.30. Irregular z-buffer. In the upper left, the view from the eye generates a set of dots at the pixel centers. Two triangles forming a cube face are shown. In the upper right, these dots are shown from the light's view. In the lower left, a shadow-map grid is imposed. For each texel a list of all dots inside its grid cell is generated. In the lower right, shadow testing is performed for the red triangle by conservatively rasterizing it. At each texel touched, shown in light red, all dots in its list are tested against the triangle for visibility by the light. (*Underlying raster images courtesy of Timo Aila and Samuli Laine [14].*)



Figure 7.31. On the left, PCF gives uniformly softened shadows for all objects. In the middle, PCSS softens the shadow with distance to the occluder, but the tree branch shadow overlapping the left corner of the crate creates artifacts. On the right, sharp shadows from IZB blended with soft from PCSS give an improved result [1711]. (Images from “Tom Clancy’s The Division,” courtesy of Ubisoft.)

Irregular Z-buffer Shadows

- Avoids sampling and bias issues with other approaches and provides perfectly sharp shadows.
- Use occluder distance to blend IZB and PCSS.